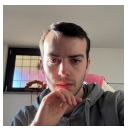
Latest updates: https://dl.acm.org/doi/10.1145/3658617.3697670

RESEARCH-ARTICLE

FactorFlow: Mapping GEMMs on Spatial Architectures through Adaptive Programming and Greedy Optimization

MARCO RONZANI, Politecnico di Milano, Milan, MI, Italy



PhD Student at Politecnico di Milano

CRISTINA SILVANO, Politecnico di Milano, Milan, MI, Italy

Open Access Support provided by:

Politecnico di Milano

PDF Download
3658617.3697670.pdf
22 March 2026
Total Citations: 2
Total Downloads: 795

Published: 04 March 2025

Citation in BibTeX format

ASPDAC '25: 30th Asia and South Pacific Design Automation Conference
January 20 - 23, 2025
Tokyo, Japan

Conference Sponsors:
SIGDA

ASPDAC '25: Proceedings of the 30th Asia and South Pacific Design Automation Conference (January 2025)
https://doi.org/10.1145/3658617.3697670
ISBN: 9798400706356

FactorFlow: Mapping GEMMs on Spatial Architectures through Adaptive Programming and Greedy Optimization

Marco Ronzani
marco.ronzani@polimi.it
DEIB, Politecnico di Milano
Milan, Italy

Cristina Silvano
cristina.silvano@polimi.it
DEIB, Politecnico di Milano
Milan, Italy

ABSTRACT

General Matrix Multiplications (GEMMs) are fundamental kernels in tensor-based scientific applications and deep learning. Modern AI accelerators using spatial architectures can run these kernels efficiently by leveraging parallelism and data reuse, but they require specific mappings to plan data movements and computations. The choice of mapping significantly impacts energy consumption and latency. Consequently, the vast space of possible mappings, unique for each GEMM-architecture pair, must be searched thoroughly to find optimal solutions. This is a complex optimization problem that imposes effective map-space exploration strategies. Current state-of-the-art mapping tools primarily address convolution kernels, a superset of GEMMs, but often fail to leverage GEMMs' specific characteristics. As a result, they struggle to consistently generate optimal mappings in a reasonable time for all GEMM-architecture pairs. This paper introduces FactorFlow, an automatic framework designed to map GEMM kernels to spatial architectures using adaptive programming and greedy optimization to minimize the energy-delay product. Our evaluation, conducted against four other state-of-the-art mapping tools on a selected set of GEMMs and architectures, demonstrates that FactorFlow consistently discovers mappings that outperform existing tools in terms of EDP while significantly reducing the exploration execution time.

KEYWORDS

Spatial architectures, DNN accelerators, GEMM mapping, adaptive programming, design space exploration

ACM Reference Format:
Marco Ronzani and Cristina Silvano. 2025. FactorFlow: Mapping GEMMs on Spatial Architectures through Adaptive Programming and Greedy Optimization. In *30th Asia and South Pacific Design Automation Conference (ASPDAC '25), January 20–23, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3658617.3697670

1 INTRODUCTION

Modern workloads like deep learning and scientific applications heavily rely on **General Matrix Multiplications (GEMMs)** and convolutions, highly parallelizable kernels with significant storage, memory bandwidth, and compute requirements. With AI driving current needs for optimizing their performance and energy consumption, the last decade has seen flourishing development, in both the industry and academia, of dedicated deep learning hardware accelerators specialized for such kernels [10, 15, 19]. A prominent design in this domain is **Spatial Architectures (SA)**, multidimensional arrays of Processing Elements (PEs) that execute Multiply and Accumulate (MAC) operations, paired with a dedicated memory hierarchy to enable data reuse. Notable examples include Eyeriss[1] and Gemmini[5] for the edge, and Google TPU[7] in datacenters.

Spatial architectures have a lot of flexibility in how they can stage data and perform operations in parallel, a specific way in which a computation is run on a SA is called **mapping**. For a pair of SA and kernel, the **map-space** is the set of all possible mappings.

It is well known that in modern computer systems, memory accesses are the dominant source of energy consumption, while memory bandwidth limitations dictate latency [1]. Thus, to best exploit SAs it is critical to identify a mapping that results in the hardware leveraging data reuse in such a way that memory accesses and bandwidth requirements are minimized. This challenge is known as the **mapping problem**, which consists of finding the best possible configuration, in terms of energy and/or latency, among those of any map-space. The problem complexity stems from the vast size of map-spaces, and a procedure capable of handling them, systematically retrieving good mappings, is called **mapper** [16].

Several mappers have already been introduced in literature for convolutions, employing techniques like random search[16], genetic algorithms[9], and simulated annealing[8]. The typical approach consists of utilizing an **analytical model** of the target SA to swiftly gather feedback on a mapping's performance and use it to guide the mapper, this is referred to as **Map-Space Exploration (MSE)**. However, while analytical models for SAs have reached a maturity level sufficient to enable MSE, the aforementioned mapping techniques remain slow. This forces mappers to compromise on sub-optimal results to reach an acceptable execution time, usually by pruning the map-space and/or early terminating the exploration. Such a trade-off is required because mapping procedures are often part of more complex toolchains (e.g. HW design-space exploration frameworks or DL compilers) [4] where a brief execution is crucial.

On the other hand, GEMMs are an extremely relevant kernel, and even more recently so due to their ample use in Transformers and large language models [10]. Convolutions are commonly converted in GEMMs as well, e.g. by Im2Col or Kn2Row [4], when they are not directly supported by the hardware. Nonetheless, since GEMMs are also a subset of convolutions, existing MSE tools focus on the latter, limiting their support for GEMMs to a byproduct. However, GEMMs shift the mapping problem's complexity from the number of dimensions involved, which decreases from six to three, to the

This work is licensed under a Creative Commons Attribution International 4.0 License.
ASPDAC '25, January 20–23, 2025, Tokyo, Japan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0635-6/25/01
https://doi.org/10.1145/3658617.3697670

larger sizes of such dimensions [10]. Consequently, current analytical models and mappers experience significant overhead when handling GEMMs and fail to leverage their different map-spaces. This leads to missed opportunities to reduce execution time and improve energy efficiency and latency in mappings. Therefore, there is ample room to improve map-space exploration for GEMMs, and with their widespread usage, any performance gains achievable by specializing a model and mapper pair for GEMMs are bound to have a significant impact.

1.1 Contributions

Our work focuses on the mapping of GEMMs onto spatial architectures, introducing and detailing three novel heuristics tailored for GEMMs that consistently retrieve near-optimal mappings while substantially reducing the mapper's execution time.

We make the following contributions:

(a) We develop **FactorFlow (FF)**, a framework comprising an analytical model and a mapper, which can find a near-optimal mapping from the specification of a SA and a GEMM.

(b) In FactorFlow, we break down the GEMMs mapping problem into sub-problems, recognize their interactions, and propose a suitable set of heuristics to address them hierarchically:
(1) Exhaustive search of loop permutations augmented with adaptive programming;
(2) Maximization of the utilization of spatial dimensions;
(3) Greedy descent factors allocation guided by a local search among adjacent mappings, guaranteeing local optimality.
Such procedures are deterministic and can potentially reach any feasible mapping, managing to exploit the full capabilities of a SA. Their effectiveness and robustness are demonstrated on the map-spaces of 4 diverse SAs and 10 GEMMs.

(c) We show that FactorFlow always finds equivalent or better mappings in less time than previous attempts by comparing it against 4 other state-of-the-art tools, where it achieves 1-161× better EDP with an up to 205× lower execution times.

1.2 State-of-the-art Analysis

Multiple tools have been proposed to map convolutions on SAs. Among them, Timeloop[16] focuses on the accuracy of its analytical model, only employing exhaustive or random search for MSE, making it slow and tying its quality of results directly to the allowed execution time. MAESTRO[11] and GAMMA[9] are a model and mapper respectively, the latter using a custom genetic algorithm to optimize mappings. MAESTRO-BLAS and FLASH[15] are instead a specialization for GEMMs of the above tools, which concentrate on heavily pruning the map-space before exhaustively searching the rest of it. CoSA[6] solves the mapping problem in one shot through a mixed integer linear programming formulation, trading modeling accuracy for speed, but often resulting in subpar mappings. ZigZag[14] is an analytical model paired with either LOMA[20] or SALSA[8] as mappers. LOMA places each prime factor of the computation dimensions on an independent loop, merges some to prune the space, and explores all their distinct orderings, only later allocating them to memory levels in a single pass, trying to maximize reuse. SALSA improves on it by using simulated annealing to reorder loops until convergence to a locally optimal order. Many

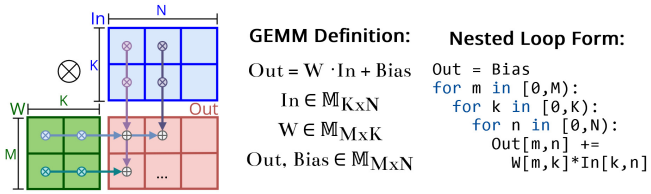


Figure 1: Definition of a GEMM and its representations.

works have also focused on optimizing GEMMs on architectures beside SAs [12, 21], e.g. by porting BLAS, and despite the differences in target hardware, their insights on GEMMs still hold here.

As previously stated, all discussed state-of-the-art tools present a shortcoming: while they can handle GEMMs, they treat them as convolutions with 3 out of 6 dimensions set to 1, resulting in modeling overheads and mapping techniques never tuned for GEMMs.

2 PRELIMINARIES

General Matrix Multiplication. By definition, a GEMM operation involves the product of two matrices with the sum of a *Bias* term, as shown in Fig. 1. From a computational perspective, a GEMM can be seen as three nested loops around a MAC operation [3]. The order of loops is arbitrary, while data reuse opportunities are now apparent as all three operands are orthogonal to a different loop and remain constant throughout it [12]. The *Bias* can be handled outside said loops by initializing *Out* with it, and carrying out the sum through the first round of accumulation. Hence, we consider as operands solely the input *In*, the weights *W*, and the output *Out*.

Spatial Architectures. They are a type of parallel computing system consisting of an **array of processing elements** supporting MAC operations that can communicate with each other and a **memory hierarchy** via interconnections varying from busses to networks-on-chip. The memory hierarchy's task is to keep PEs supplied with data, minimizing the stall cycles they incur and the energy expended to access subsequent values. To this end SAs are built with small, fast, memories near PEs, usually register files, going up to intermediate on-chip buffers (e.g. scratchpads), and finally relying on the large and slow off-chip memory (e.g. DRAM).

The key performance metrics of a SA on a given workload are the energy it consumes, its latency, and the Energy-Delay Product (EDP), which are mostly dominated by the cost of accessing the slowest levels of the memory hierarchy. As an example, a single access to off-chip DRAM has been shown to consume more energy than 200 reads from a PE's registers or 500 MAC operations, furthermore, DRAM bandwidth can be less than a tenth of on-chip SRAM buffers, impacting latency as well [1]. Consequently, SAs exploit **data reuse** to minimize costly memory accesses, reading a value once from off-chip memory and accessing it multiple times through on-chip memories before offloading it once more. The Mechanisms enabling reuse are multicast and reduction, which can be either temporal, across successive MACs, or spatial, across PEs [11].

In this work we abstract a SAs as a hierarchy of three types of components, following the same modeling scheme of [11, 16]:

- **Memory Level**, representing a storage component holding part of a GEMM's three operands, described by its capacity, access energy, bandwidth, and which operand(s) it stores.

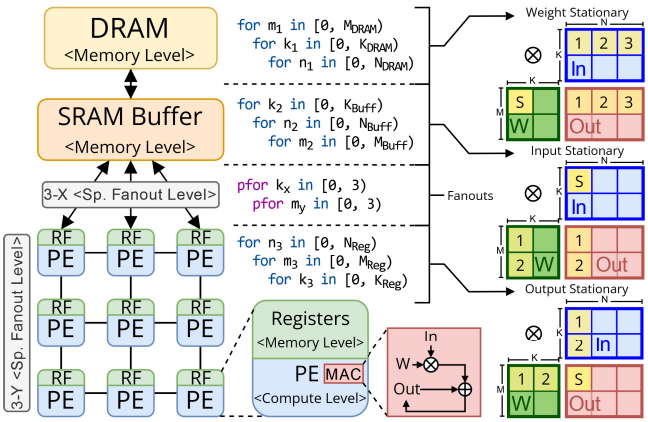


Figure 2: Model of a SA composed of a 2D mesh of PEs with a GEMM mapping and dataflows outline. "pfor" denotes spatial iterations.

- **Spatial Fanout Level**, modeling the spatial replication of all subsequent components in a mesh of instances. Its key characteristics are the number of instances available and any spatial multicast/reduction capabilities.
- **Compute Level**, representing a functional unit capable of performing a MAC operation, defined by the energy and the number of clock cycles required for a MAC.

2.1 Mapping GEMMs on Spatial Architectures

Given a SA and a GEMM, the goal of a **mapping** is to determine the allocation of operands throughout the memory hierarchy and the binding of each MAC operation to a PE. The set of all possible mappings is the **map-space**. To be usable, a mapping must be valid, meaning that it must respect constraints both derived from limited hardware resources and specified by the user. Therefore, the **mapping problem** is a constrained optimization problem, and solving it means finding a procedure to search the map-space for a valid and optimal mapping w.r.t energy and/or latency [16]. Let us now introduce the three main decisions involved in a mapping [11]:

(1) Tiling. To fit a GEMM's dimensions on the limited hardware resources of a SA, we repeatedly tile it into progressively smaller GEMMs throughout the SA's memory hierarchy. Tile sizes are chosen to both fit in and exploit to the maximum the storage available at their level. This is represented by replicating the three nested loops of a GEMM across all memory levels of the SA, with each copy performing a subset of the total iterations along every dimension. Loops on a memory level unfold over time, and each memory in the hierarchy only needs to store all the values used in iterations on its loops or inner ones, enabling their reuse across all such iterations.

(2) Parallelism Strategy. Spatial (sp.) fanout levels allow some iterations to unfold spatially across multiple instances of subsequent levels, each running the same inner loops concurrently on partially different data. Hardware limitations on resources and flexibility constrain the maximum number of such spatial iterations and also which dimensions can exploit them. Thus, a mapping must specify which dimension(s) leverage parallelism and allocate their spatial iterations. This is modeled like tiling, by having a copy of one or more of a GEMM's loops dedicated to each sp. fanout level.

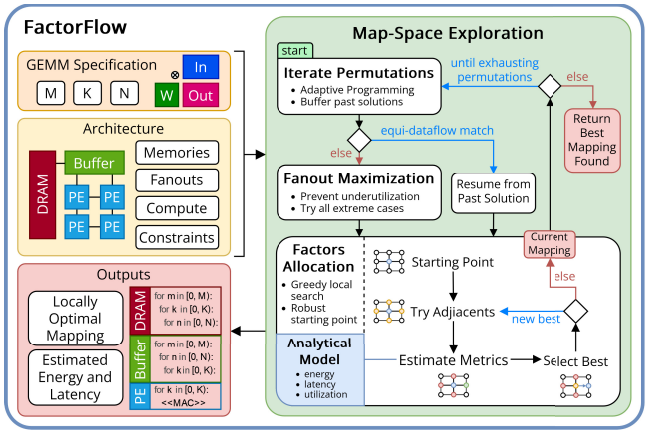


Figure 3: Overview of FactorFlow.

(3) Loop Ordering. Any group of three loops within a tiled GEMM can be arbitrarily reordered without affecting the correctness of the computation, but with relevant implications on data movements and reuse within the SA. We call **dataflow** a specific order of a loop triplet and the data access patterns derived from it, classifying them according to the innermost iterated dimension [1].

In a *weight stationary* dataflow the output's width, N, is iterated first, resulting in weights reuse (multicast) across the innermost iterations. For an *input stationary* dataflow it is the output's height, M, resulting in inputs reuse (multicast). While an *output stationary* dataflow iterates first on the GEMM's inner dimension, K, enabling the in-place reduction of outputs. In summary, each dataflow reuses the operand orthogonal to the dimension of the innermost iterations it performs, ignoring loops with a single iteration.

Notably, while tiling enables reuse, the dataflow determines whether an operand is actually reused on a memory level or not.

The Map-Space. Constituted by all possible combinations of the three above mapping decisions, each map-space is discrete, finite, and extremely vast. With GEMMs having 3 often large dimensions, tiling is the major source of mappings. Furthermore, optimal mappings are scarce, and the variation in EDP across the map-space can exceed $10^4\times$ [10]. E.g., a GEMM with all dimensions set to 2^{10} can be mapped in about 10^{12} valid ways on Gemmini, the simplest SA we considered, with only around 10^2 mappings having the best EDP.

To better represent and navigate the map-space, this work decomposes each GEMM dimension in its **prime factors**, and those become the atomic elements assigned to the different loops throughout the map-space. For concision, this work will proceed to use the terms "factor" and "prime factor" interchangeably.

3 FACTORFLOW

FactorFlow's implementation follows current state-of-the-art practices and comprises an **analytical model** to estimate a mapping's resulting energy and latency and a **mapper** that employs heuristics to carry out MSE. Both have been realized in Python [22].

3.1 Analytical Model

FactorFlow introduces its own analytical model focused on GEMMs, which enables us to move from the map-space to the space of performance metrics. On the set of considered architectures (see

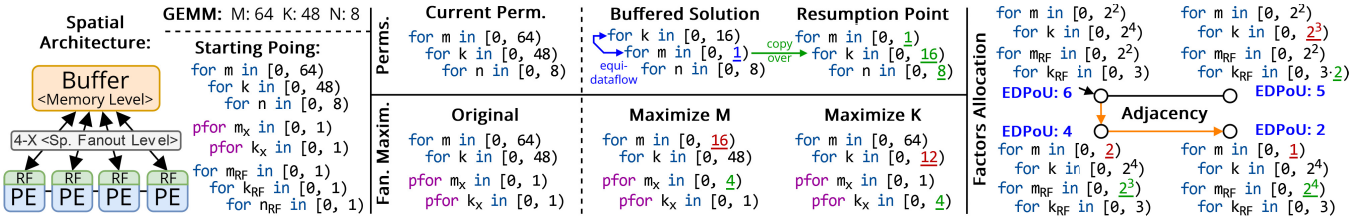


Figure 4: Map Space Exploration Steps Examples.

Table 1), such a model is functionally equivalent to Timeloop and, as a result, it reaches the same average estimation error of 5% [16].

The architectural model is fully flexible and expressed via components reflecting the description in Sec. 2, with memories modeled after the Buffet paradigm [17]. Additionally, FF supports operands skipping a memory level (bypass), multiple dimensions mapped on the same sp. fanout level, and padding. The model is based on the evaluation of, in order: tile sizes, memory operations, and latency, through 3 passes over the architecture's hierarchy. Each pass analytically derives its target metric(s) starting from each level's type, factors, dataflow, and the results from previous passes.

Being built from the ground up for GEMMs, FF's model is more lightweight than those focused on convolutions, needing to attend to fewer dimensions. As a result, throughout all our experiments, FF always manages to evaluate a mapping in less than 1 ms.

To acquire an estimation of the energy consumption of all modeled hardware components, FF relies on Accelergy[23].

3.2 Map-Space Exploration

To build a mapper, FF addresses mapping decisions in order, recognizing that permutations are comparable once their optimal sp. fanouts and tilings are known, sp. fanouts can be initialized to maximize instances utilization, and tilings can be compared and co-optimized with sp. fanouts in greedy moves for fixed permutations. Henceforth, we introduce heuristics to perform those three steps: *(1) iterate permutations* tries all loop orders, sending each to *(2) fanout maximization* which saturates spatial iterations, lastly *(3) greedy descent factors allocation* explores the neighborhood of each mapping, reaching a local optimum. Such steps are performed hierarchically, meaning that completion of each step might require multiple executions of subsequent ones. Each encountered mapping has its performance metrics estimated via the analytical model. From these, we derive a single compound optimization metric to rank each mapping, intended to capture overall performance and comprising energy, latency, and instances utilization ratio:

$$EDPoU = \frac{Energy \cdot Latency}{Utilization} \tag{1}$$

The objective of map-space exploration is to minimize the $EDPoU$.

In this optimization problem, constraints can derive from finite hardware resources or be user-provided. Enforcing the latter type can shrink the map-space and ease MSE. FF leverages this; however, for brevity, user-provided constraints will not be hereby considered.

(1) Iterate Permutations. Loop permutations on all memory levels are exhaustively searched. With 3 loops per level and m memory levels in the SA, total permutations are 6^m, a manageable number only for simple SAs and assuming the efficiency of subsequent steps. However, not all permutations have to be evaluated from scratch.

The exploration of specific permutations is sped up leveraging **adaptive programming** [13]: if a permutation differs from any already tried one by a change solely in the position of loops that have a single assigned iteration in the best mapping found for the already tried permutation, we refer to the pair of permutations as *equi-dataflow*. When equi-dataflow pairs are encountered the current permutation skips the *fanout maximization* step and resumes the *greedy descent factors allocation* step from the best allocation of prime factors found for the past permutation. In any other case, a permutation is simply forwarded to the *fanout maximization* step.

This is feasible since changing the position of loops with a single iteration does not alter the mapping's behavior, dataflows are preserved and so is the solution's $EDPoU$, but the new order of loops might lead to some further possibility for optimization. That said, only a few, if any, new *greedy descent* moves are needed to re-optimize a mapping, saving as much time as skipping permutations entirely, but without pruning the map-space. Memory levels are permuted in lexicographical order, innermost to outermost, therefore we only need to check for an equi-dataflow pair on the last permuted level, disregarding inner loops. In the case of a match, then, all inner loop orders are also set to those of the already known mapping, assumed to be optimal, and leaving no further need to try their permutations. Using adaptive programming requires buffering of past solutions, but this way at most one solution per permutation of a level needs to be stored, totaling at most at 6 per memory level.

This step can be optionally run in parallel by distributing subsets of all permutations across multiple concurrent tasks, almost linearly reducing the execution time with the task count.

(2) Fanout Maximization. Having fixed the permutations, we maximize the utilization of the SA's PEs. This step allocates to each sp. fanout level as many prime factors taken from its assigned GEMM's dimension as can fit on its number of instances. It starts by assigning the prime factors that are shared between the dimension and the number of instances and concludes with the combination of disjoint leftover factors that maximizes utilization. Optionally, GEMM dimensions can be padded to an exact multiple of a sp. fanout's instances, enabling full exploitation of the SA's parallelism.

In case of sp. fanouts which can be used by multiple dimensions of a GEMM at the same time (e.g. this is the case in Eyeriss), FF adopts countermeasures to ensure that all possible parallelism strategies are accounted for. First, multiple mappings are prepared during *fanout maximization*, by targeting once every candidate GEMM dimension, and each goes independently through the next step. Second, the *factors allocation* step is tasked to co-optimize spatial and temporal iterations, swapping factors across dimensions sharing the same fanout whenever it is deemed beneficial.

This technique is based on the intuition that SAs achieve their best performance only when their parallelism capabilities are fully

exploited. While this is trivially proven in the case of sp. fanouts assigned to a single dimension, where spatial reuse always improves, we circumvent the issue of verifying whether maximizing only one of multiple candidate dimensions for a sp. fanout is optimal by allowing the next step to refine such a choice.

(3) Greedy Descent Factors Allocation. The allocation of factors to memory levels addresses the tiling part of a mapping.

First, we define two mappings as *adjacent* if their factors allocations differ solely by the position of a single factor along one dimension. In other words, moving a single factor across any two levels transitions between adjacent mappings.

In this step a **local search** [13] is conducted by following adjacency. Starting from an initial valid allocation, with fixed permutations, all never-tried feasible adjacent mappings are evaluated, and the best one by *EDPoU* is picked as the next starting point (greedy move). The routine repeats until no new adjacent mappings better than the current one are found, guaranteeing that the present solution is a local optimum w.r.t. the adjacency relation.

The actual implementation of this procedure is configurable in the number of adjacency hops N it explores before selecting the best next starting point, e.g. with $N = 2$ for any adjacent mapping, all its adjacent ones are also considered, except for any already seen one. This way FF can skip over intermediate worse mappings and reach better distant ones. Increasing N trades off FF's execution time for its quality of results. By default, $N \leq 2$, but higher values are justifiable if execution time is not a concern.

Local Search Starting Point. Navigating the map-space via N hops of the adjacency relation implies that proximity to constraints, e.g. when multiple memory levels are at capacity, reduces the number of available feasible moves, making local optima more frequent. Additionally, the best mappings are inevitably close to constraints, as they tend to fully utilize hardware resources to raise reuse. Since the proposed *greedy descent* stops on the first local optimum it finds, to encourage a thorough search of the map-space it is essential to pick a starting point far from any local optima, but also one still allowing full reachability of near-constraints areas where a global optimum might lie. Therefore, to establish a robust heuristic, the default starting point in the *factors allocation* step places all available factors on the first memory level's loops, the map-space's point furthest from constraints and with the most valid adjacent mappings.

To empirically prove our reasoning, we compared the proposed starting point with 10^4 distinct random ones for all examined GEMM-SA pairs (see Tables 1, 2) on fixed loop permutations, noting that our choice has always allowed the best solution to be reached.

Spatial and Temporal Iterations Co-Optimization. The *factors allocation* step can optionally co-optimize spatial and temporal iterations (as required in step (2)) by also targeting sp. fanout levels, allowing factors to freely move across any pair of loops as long as the mapping improves. To such end, setting $N \geq 2$ is required, because to alter spatial iterations an adjacent mapping must first remove a factor from a fanout level, inevitably worsening its utilization, with only a second hop possibly improving upon the original mapping.

Preventing Underutilization. The *fanout maximization* step is an essential setup to ensure a robust *greedy descent*: it removes the risk of stopping on local optima with poor utilization caused by disjoint primes between sp. fanout instances and a GEMM dimension occupying most of a sp. fanout level. Such a case is common with greedy moves, which would try to immediately increase utilization by moving the largest factor available first, regardless of it being shared or disjoint. However, that is undesirable, since only shared factors can reach complete utilization, but unfortunately, once factors are allocated to sp. fanouts, it is often impossible for the *factors allocation* step to find a sequence of greedy moves swapping them out. While this can be mitigated by raising N, the resulting execution time penalty makes *fanout maximization* the better option.

Name	On-chip Mem	# Mems	Sp. Fanouts	Sp. Fanout Dims	Target
Gemmini[5]	576 KiB	4	16, 16	M, K	Edge
Eyeriss[1]	196 KiB	5	14, 12	M/K, M	Edge
Simba[18]	752 KiB	6	16, 4, 4	M/K, M, K	Datacenter
TPUv1[7]	30 MiB	6	256, 256	M, K	Datacenter

Table 1: Features of the selected spatial architectures.

GEMM	BERT Large [2]				Scientific Applications [15]					
Dimension	I	II	III	IV	V	VI	VII	VIII	IX	X
M	3072	4096	64	4096	8192	1024	8	8	8192	512
K	1024	64	4096	1024	8192	8192	8192	1024	1024	256
N	4096	4096	4096	4096	8192	1024	8	8192	8	256

Table 2: Sizes of the GEMMs used in the experiments.

SA	Cumulative Execution Time per Step					# tried	# equi-df.
	(3)	(2)	(1)-1ts	(1)-8ts	(1)-8ts-nAP	perms.	matches
Gemmini[5]	0.021 s	0.021 s	1.298 s	0.634 s	1.558 s	216	196
Eyeriss[1]	0.011 s	0.022 s	4.042 s	0.788 s	48.270 s	15552	15532
Simba[18]	0.120 s	0.242 s	40.021 s	4.250 s	6350.485 s	93312	93292
TPUv1[7]	0.135 s	0.136 s	4.323 s	2.740 s	749.586 s	7776	7774

·**ts**: number of concurrent tasks used, **nAP**: adaptive programming disabled.

Table 3: FactorFlow execution time breakdown. GEMM: I (Tab. 2).

4 EXPERIMENTAL RESULTS

To demonstrate FF's effectiveness, we compare it against 4 state-of-the-art mapping tools and the global optimum across 40 diverse map-spaces, derived from 4 literature spatial architectures and 10 GEMMs representative of AI workloads and scientific applications.

4.1 Experimental Setup

As the target architectures (see Table 1) we selected Gemmini[5] and the TPUv1 (8-bit mode) [7] as representatives of systolic-array-based designs targeting edge and datacenter environments respectively. Then, Eyeriss[1] was chosen due to its significance in the field, and, together with Simba (single chiplet) [18], both present reconfigurable dataflows, requiring suitable parallelism strategy handling from mappers. Selected GEMMs (see Table 2) include 4 from the BERT Transformer (large version) [2], with sequence length set to 4096, and 6 from the previous study performed in [15]. Compared tools were instead chosen to cover a wide range of map-space exploration techniques: random search with Timeloop[16], constrained optimization with CoSA[6], loop ordering-based with LOMA[20], and simulated annealing with SALSA[8] [1].

To enable a fair comparison of mapping techniques and overcome the modeling differences across tools, we evaluate each mapping on FF and Timeloop's analytical models, with both always agreeing on the presented metrics. All compared tools, however, could only use their internal model during MSE, hence all SAs were carefully modeled with equivalent specifications in each of them. Model compatibility with LOMA's uneven mappings [14] has been achieved in

[1]GAMMA[9] could not be compared due to its lack of model flexibility.
No publicly available version of MAESTRO-BLAS and FLASH [15] was instead found.

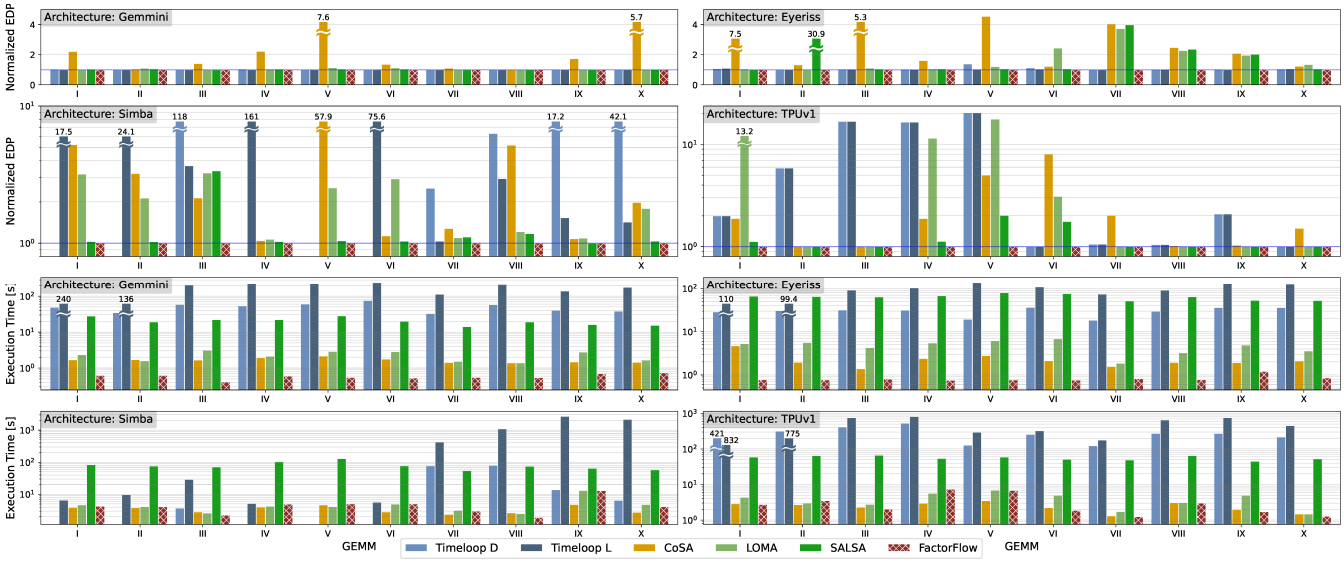


Figure 5: EDP and Execution Time for FactorFlow compared to 4 state-of-the-art mapping tools.

FF and Timeloop via additional SA levels bearing no impact on performance. Each mapper was used with its default settings, except Timeloop, for which "D" denotes the default configuration, while "L" indicates that it was allowed to try 15× more mappings before terminating. In FF, step (3), N was set to 1 for Gemmini and 2 otherwise. For every mapper, the target metric was set to be the EDP or an equivalent one. Note that missing data points always indicate the tool's failure to find a valid mapping. Additionally, Timeloop was used to perform an exhaustive search of all map-spaces, for each finding a global optimum. The execution time was measured on an AMD Ryzen 3900xt @ 3.8GHz and averaged over 5 runs without significant variance. Parallel execution across 24 logical cores was enabled for FF and for all other mapping tools.

4.2 Comparison Results

Across all SAs and GEMMs, as shown in Fig. 5, FactorFlow consistently achieves near-optimal mappings in at most 13.1 s and 2.3 s on average. On simpler map-spaces (Gemmini, Eyeriss) FF is always tied with other tools on the global optimum, but reaches it in 1.6-143.8× less time. Whereas, on larger map-spaces (Simba, TPUv1) FF finds the best mapping every time, averaging a 7.5× EDP reduction, while generally leading in execution time. Timeloop's random sampling often lands close to top mappings, but fails to notice, wasting effort on a large number of attempts. FF's local search obviates such an issue, improving EDP by 1-161.1× and execution time by 1.5-204.8×. To simultaneously explore loop orders and tilings in a reasonable time, LOMA and SALSA require pruning and simulated annealing, respectively. Instead, by treating the two decisions individually, FF's heuristics reach a 1-30.9× lower EDP in up to 104.2× less time. CoSA's mapper solves a linear programming problem, granting it consistent execution times, but penalizing its SA modeling capabilities, therefore FF always achieves 1-57.9× better mappings, with frequently faster execution times despite running an actual exploration. At last, to study the correctness of the proposed heuristics, we found that FF's mappings match the global optimum in 36/40 experiments, with an EDP at most 2.7% higher in the rest.

In Table 3 we report the average execution time required by each step of FF, alongside the total number of explored permutations. The critical role of adaptive programming, with the likely occurrence of equi-dataflow permutations, is evident, since disabling it increases execution time by 2.5-1494.2×. Moreover, found mappings were the same in our trials with and without adaptive programming.

Going back to convolutions: unless they are lowered into GEMMs, the proposed techniques cannot apply to them as is. With 6 nested loops for m memory levels, their total possible permutations reach 720^m, rendering exhaustive search impossible. Adaptive programming may need to buffer a costly $720m$ solutions as well. Furthermore, using $N \leq 2$ during local search is only justified when moving 1 or 2 factors across memories can improve reuse, but this fails on convolutions, where multiple dimensions need to be acted on to affect reuse, requiring higher Ns that also worsen execution time.

5 CONCLUSION

In this work, we introduced FactorFlow, a MSE tool specifically designed for mapping GEMMs on spatial architectures using three novel heuristics. Our approach leverages an exhaustive exploration paired with adaptive programming, followed by a greedy optimization routine with a suitable starting point, which grant it efficiency and robustness. Obtained mappings are guaranteed to be a local optimum, which we have empirically shown to often coincide with a global one. Our experimental results demonstrated FactorFlow's effectiveness, as it consistently outperformed other state-of-the-art solutions in both mapping energy-delay product and mapper execution time across many diverse map-spaces. As the development and deployment of spatial architectures continue, steered by the needs of AI, the mapping problem will become a recurring challenge that must be repeatedly addressed to fully exploit spatial architectures on any new workload. In this landscape, our study advanced mapping techniques for the crucial GEMM kernel. Future work will aim to refine our approach and explore other optimization strategies.

Alongside this publication, FactorFlow has been released as an open-source artifact at: https://github.com/EMJzero/FactorFlow.

ACKNOWLEDGEMENTS

This work has been partially supported by the Spoke 1 on *Future HPC* of the Italian Research Center on High-Performance Computing, Big Data and Quantum Computing (ICSC) funded by MUR Mission 4 - Next Generation EU.

REFERENCES

[1] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 367–379. https://doi.org/10.1109/ISCA.2016.40

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL]

[3] J. J. Dongarra, Jeremy Du Croz, Sven Hammarling, and I. S. Duff. 1990. A set of level 3 basic linear algebra subprograms. *ACM Trans. Math. Softw.* 16, 1 (mar 1990), 1–17. https://doi.org/10.1145/77626.79170

[4] Fabrizio Ferrandi, Serena Curzel, Leandro Fiorin, Daniele Ielmini, Cristina Silvano, Francesco Conti, Alessio Burrello, Francesco Barchi, Luca Benini, Luciano Lavagno, Teodoro Urso, Enrico Calore, Sebastiano Fabio Schifano, Cristian Zambelli, Maurizio Palesi, Giuseppe Ascia, Enrico Russo, Nicola Petra, Davide De Caro, Gennaro Di Meo, Valeria Cardellini, Salvatore Filippone, Francesco Lo Presti, Francesco Silvestri, Paolo Palazzari, and Stefania Perri. 2023. A Survey on Design Methodologies for Accelerating Deep Learning on Heterogeneous Architectures. arXiv:2311.17815 [cs.AR]

[5] Hasan Genc, Seah Kim, Alon Amid, Ameer Haj-Ali, Vighnesh Iyer, Pranav Prakash, Jerry Zhao, Daniel Grubb, Harrison Liew, Howard Mao, Albert Ou, Colin Schmidt, Samuel Steffl, John Wright, Ion Stoica, Jonathan Ragan-Kelley, Krste Asanovic, Borivoje Nikolic, and Yakun Sophia Shao. 2021. Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 769–774. https://doi.org/10.1109/DAC18074.2021.9586216

[6] Qijing Huang, Minwoo Kang, Grace Dinh, Thomas Norell, Aravind Kalaiah, James Demmel, John Wawrzynek, and Yakun Sophia Shao. 2021. CoSA: Scheduling by Constrained Optimization for Spatial Accelerators. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 554–566. https://doi.org/10.1109/ISCA52012.2021.00050

[7] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. *SIGARCH Comput. Archit. News* 45, 2 (jun 2017), 1–12. https://doi.org/10.1145/3140659.3080246

[8] Victor J.B. Jung, Arne Symons, Linyan Mei, Marian Verhelst, and Luca Benini. 2023. SALSA: Simulated Annealing based Loop-Ordering Scheduler for DNN Accelerators. In *2023 IEEE 5th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. 1–5. https://doi.org/10.1109/AICAS57966.2023.10168625

[9] Sheng-Chun Kao and Tushar Krishna. 2020. GAMMA: Automating the HW Mapping of DNN Models on Accelerators via Genetic Algorithm. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.

[10] Sehoon Kim, Coleman Hooper, Thanakul Wattanawong, Minwoo Kang, Ruohan Yan, Hasan Genc, Grace Dinh, Qijing Huang, Kurt Keutzer, Michael W. Mahoney, Yakun Sophia Shao, and Amir Gholami. 2023. Full Stack Optimization of Transformer Inference: a Survey. arXiv:2302.14017 [cs.CL]

[11] Hyoukjun Kwon, Prasanth Chatarasi, Michael Pellauer, Angshuman Parashar, Vivek Sarkar, and Tushar Krishna. 2019. Understanding Reuse, Performance, and Hardware Cost of DNN Dataflow: A Data-Centric Approach. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 754–768. https://doi.org/10.1145/3352460.3358252

[12] Tze Meng Low, Francisco D. Igual, Tyler M. Smith, and Enrique S. Quintana-Orti. 2016. Analytical Modeling Is Enough for High-Performance BLIS. *ACM Trans. Math. Softw.* 43, 2, Article 12 (aug 2016), 18 pages. https://doi.org/10.1145/2925987

[13] Rafael Martí, Mauricio G.C. Resende, and Celso C. Ribeiro. 2013. Multi-start methods for combinatorial optimization. *European Journal of Operational Research* 226, 1 (2013), 1–8. https://doi.org/10.1016/j.ejor.2012.10.012

[14] Linyan Mei, Pouya Houshmand, Vikram Jain, Sebastian Giraldo, and Marian Verhelst. 2021. ZigZag: Enlarging Joint Architecture-Mapping Design Space Exploration for DNN Accelerators. *IEEE Trans. Comput.* 70, 8 (2021), 1160–1174. https://doi.org/10.1109/TC.2021.3059962

[15] Gordon E. Moon, Hyoukjun Kwon, Geonhwa Jeong, Prasanth Chatarasi, Sivasankaran Rajamanickam, and Tushar Krishna. 2021. Evaluating Spatial Accelerator Architectures with Tiled Matrix-Matrix Multiplication. arXiv:2106.10499 [cs.DC]

[16] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W. Keckler, and Joel Emer. 2019. Timeloop: A Systematic Approach to DNN Accelerator Evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 304–315. https://doi.org/10.1109/ISPASS.2019.00042

[17] Michael Pellauer, Yakun Sophia Shao, Jason Clemons, Neal Crago, Kartik Hegde, Rangharajan Venkatesan, Stephen W. Keckler, Christopher W. Fletcher, and Joel Emer. 2019. Buffets: An Efficient and Composable Storage Idiom for Explicit Decoupled Data Orchestration. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) *(ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 137–151. https://doi.org/10.1145/3297858.3304025

[18] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, Stephen G. Tell, Yanqing Zhang, William J. Dally, Joel Emer, C. Thomas Gray, Brucek Khailany, and Stephen W. Keckler. 2019. Simba: Scaling Deep-Learning Inference with Multi-Chip-Module-Based Architecture. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 14–27. https://doi.org/10.1145/3352460.3358302

[19] Cristina Silvano, Daniele Ielmini, Fabrizio Ferrandi, Leandro Fiorin, Serena Curzel, Luca Benini, Francesco Conti, Angelo Garofalo, Cristian Zambelli, Enrico Calore, Sebastiano Fabio Schifano, Maurizio Palesi, Giuseppe Ascia, Davide Patti, Stefania Perri, Nicola Petra, Davide De Caro, Luciano Lavagno, Teodoro Urso, Valeria Cardellini, Gian Carlo Cardarilli, and Robert Birke. 2023. A Survey on Deep Learning Hardware Accelerators for Heterogeneous HPC Platforms. arXiv:2306.15552 [cs.AR]

[20] Arne Symons, Linyan Mei, and Marian Verhelst. 2021. LOMA: Fast Auto-Scheduling on DNN Accelerators through Loop-Order-based Memory Allocation. In *2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. 1–4. https://doi.org/10.1109/AICAS51828.2021.9458493

[21] Guangming Tan, Linchuan Li, Sean Triechle, Everett Phillips, Yungang Bao, and Ninghui Sun. 2011. Fast implementation of DGEMM on Fermi GPU. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis* (Seattle, Washington) *(SC '11)*. Association for Computing Machinery, New York, NY, USA, Article 35, 11 pages. https://doi.org/10.1145/2063384.2063431

[22] Guido Van Rossum and Fred L Drake Jr. 1995. *Python tutorial*. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands.

[23] Yannan Nellie Wu, Joel S. Emer, and Vivienne Sze. 2019. Accelergy: An Architecture-Level Energy Estimation Methodology for Accelerator Designs. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8. https://doi.org/10.1109/ICCAD45719.2019.8942149